

Administración de Sistemas Operativos

UD – 1

Script Bash

Contenidos







ÍNDICE

¿Qué es Bash? **3**

¿Por qué aprender scripting Bash en ASIR? **3**

Estructura básica de un script **4**

Variables de entorno comunes en administración *4*
Variables de usuario *4*
Parámetros posicionales *5*
Parámetros Especiales *5*
Comillas: simples, dobles y escape *5*

Redireccionamiento y Tuberías **6**

Redireccionamiento *6*
Tuberías *6*

Estructuras Condicionales **7**

Comando Test *7*
Operadores lógicos *7*
Estructura IF *8*
Estructura Case *8*

Estructuras de Repetición (Bucles) **9**

Bucle for *9*
Bucle While *9*
Control de bucles *9*

Funciones **10**

Definición y llamada *10*
Parámetros *10*
Retorno de valores *10*

Plantilla de script profesional **11**

Descripción o cabecera del script *11*
Funciones *11*
Zona de Validación *11*
Principal *12*

1. ¿QUÉ ES BASH?

Bash (Bourne-Again SHell) es un intérprete de comandos y lenguaje de scripting ampliamente utilizado en sistemas Linux y Unix. Es el shell por defecto en la mayoría de distribuciones Linux y proporciona una interfaz tanto interactiva como de programación para automatizar tareas administrativas .

Bash es tanto un intérprete de comandos (ejecuta órdenes escritas en terminal) como un lenguaje de programación (permite variables, condicionales, bucles y funciones). Su sintaxis está optimizada para la manipulación de archivos y procesos, lo cual lo hace ideal para la administración de sistemas .



2. ¿POR QUÉ APRENDER SCRIPTING BASH EN ASIR?

En el módulo de Administración de Sistemas Operativos, el scripting es una competencia fundamental porque permite:

- **Automatizar** tareas repetitivas: copias de seguridad, gestión de usuarios, monitorización de servicios.
- **Administrar** servicios del sistema: manipular ficheros de configuración, reiniciar demonios, procesar logs.
- **Integrar** herramientas: combinar comandos del sistema en flujos de trabajo complejos.
- **Documentar** procedimientos: los scripts sirven como documentación ejecutable de procesos administrativos .



3. ESTRUCTURA BÁSICA DE UN SCRIPT

Todo script Bash comienza con la línea shebang, que indica al sistema qué intérprete debe ejecutarlo:

```
#!/bin/bash
# Este es un comentario
echo "Hola, mundo"
```

La línea #!/bin/bash debe ser la primera del archivo. Los comentarios comienzan con # .

Los scripts se crean con cualquier editor de texto (vim, nano, etc.). Para que un script sea ejecutable, se le deben asignar permisos de ejecución:

```
chmod +x mi_script.sh
./mi_script.sh
```

3.1 Variables de entorno comunes en administración

Una variable de entorno es un valor dinámico con nombre que está guardado dentro del sistema operativo o del contexto de un proceso. Funciona como una especie de "pizarra global" o nota adhesiva a la que cualquier programa o script en ejecución puede acceder para leer datos de configuración sin necesidad de que esa información esté escrita directamente (hardcodeada) en su código fuente.

$HOME	→	*Directorio personal del usuario*
$USER	→	*Nombre del usuario actual*
$PATH	→	*Rutas de búsqueda de ejecutables*
$HOSTNAME	→	*Nombre del equipo*
$PWD	→	*Directorio de trabajo actual*
$?	→	*Código de salida del último comando*

3.2 Variables de usuario

Las variables se asignan sin espacios alrededor del “=”, Para usar una variable, se antepone “$” :

```
SERVIDOR="192.168.1.10"
PUERTO=22
```

```
echo "Conectando a $SERVIDOR en el puerto $PUERTO"
```

Aritmética en Bash

Sintaxis	Descripción
`$((expresión))`	Evaluación aritmética
`let variable=expresión`	Asignación aritmética
`variable=$((variable + 1))`	Incremento

3.3 Parámetros posicionales

Los scripts pueden recibir argumentos desde la línea de comandos. Bash los expone mediante parámetros posicionales :

$0 → *Nombre del script*

$1, $2, … → *Primer argumento, segundo argumento, etc.*

$# → *Número total de argumentos*

$* → *Todos los argumentos como una sola cadena*

$@ → *Todos los argumentos como elementos separados*

```
#!/bin/bash
echo "Script: $0"
echo "Primer argumento: $1"
echo "Total de argumentos: $#"
```

3.4 Parámetros Especiales

Otros parámetros que son importantes y debes conocer:

$? → *Código de salida del último comando* **(0 = éxito, distinto de 0 = error).**

$$ → *PID del proceso actual.*

$! → *PID del último proceso en segundo plano.*

3.5 Comillas: simples, dobles y escape

En Bash, la diferencia principal entre las comillas simples (') y las comillas dobles (") radica en cómo tratan los caracteres especiales, especialmente el signo de dólar ($) utilizado para las variables.

Aquí tienes la regla de oro:Las comillas simples son estrictas:

- Todo lo que pongas dentro se interpreta literalmente como texto plano.

`echo '$HOME'` → `$HOME`

- Las comillas dobles son flexibles: Permiten la expansión de variables, la ejecución de comandos y el uso de caracteres de escape.

`echo "$HOME"` → `/home/usuario`

4. REDIRECCIONAMIENTO Y TUBERíAS

4.1 Redireccionamiento

El redireccionamiento en Linux es una característica de la terminal que permite desviar la entrada o la salida de un comando hacia un archivo, un dispositivo o incluso hacia otro comando.

Operador	Significado	Ejemplo
`>`	Redirige stdout (sobrescribe)	`ls > lista.txt`
`>>`	Redirige stdout (añade)	`date >> registro.txt`
`2>`	Redirige stderr	`comando 2> errores.txt`
`&>`	Redirige stdout y stderr	`script.sh &> salida.txt`
`<`	Redirige stdin desde archivo	`wc -l < datos.txt`

4.2 Tuberías

La tubería | conecta la salida de un comando con la entrada del siguiente. Es una de las herramientas más poderosas en administración de sistemas:

```
ps aux | grep apache | wc -l
```

5. ESTRUCTURAS CONDICIONALES

5.1 Comando Test

El comando test (o su alias [) evalúa condiciones. Devuelve 0 (éxito) si la condición es verdadera, o distinto de 0 si es falsa.

Operador	Significado
`-f archivo`	Verdadero si es un archivo regular
`-d archivo`	Verdadero si es un directorio
`-r archivo`	Verdadero si tiene permiso de lectura
`-w archivo`	Verdadero si tiene permiso de escritura
`-x archivo`	Verdadero si tiene permiso de ejecución
`-z cadena`	Verdadero si la cadena está vacía
`-n cadena`	Verdadero si la cadena no está vacía
`num1 -eq num2`	Igualdad numérica
`num1 -lt num2`	Menor que
`num1 -gt num2`	Mayor que
`cad1 = cad2`	Igualdad de cadenas
`cad1 != cad2`	Desigualdad de cadenas

5.2 Operadores lógicos

Los operadores lógicos se utilizan para combinar múltiples condiciones dentro de tus estructuras de control (`if`, `while`, etc.)

Operador	Significado	Ejemplo
`&&`	Y lógico	`[ -f a ] && [ -r a ]`
`||`	O lógico	`[ -f a ] || [ -f b ]`
`!`	Negación	`[ ! -f archivo ]`

5.3 Estructura IF

La estructura if en Bash se utiliza para tomar decisiones dentro de un script. A diferencia de otros lenguajes de programación tradicionales, en Bash el if evalúa el código de salida (exit status) de un comando o una condición. Si el resultado es cero (0), significa éxito y la condición se cumple.

```
if [ -f /etc/passwd ]; then
    echo "El archivo existe"
else
    echo "El archivo NO existe"
fi
```

```
if [ "$1" = "start" ]; then
    echo "Iniciando servicio..."
elif [ "$1" = "stop" ]; then
    echo "Deteniendo servicio..."
else
    echo "Uso: $0 {start|stop}"
    exit 1
fi
```

5.4 Estructura Case

La estructura case es la alternativa ideal al uso excesivo de bloques if-elif-else cuando necesitas evaluar una misma variable frente a múltiples opciones o patrones. Es el equivalente al switch-case de otros lenguajes de programación:

```
read -p "Introduce el número de tu opción: " OPCION

case $OPCION in
    1)
        echo "Iniciando copia de seguridad..."
        # Aquí iría el comando de respaldo
        ;;
    2)
        echo "Buscando actualizaciones..."
        sudo apt update && sudo apt upgrade -y
        ;;
    3)
        echo "Saliendo del programa. ¡Adiós!"
        exit 0
        ;;
    *)
        echo "Opción no válida. Por favor, elige 1, 2 o 3."
        ;;
esac
```

6. ESTRUCTURAS DE REPETICIÓN (BUCLES)

6.1 Bucle for

El bucle for itera sobre una lista de elementos:

```
#!/bin/bash
for usuario in ana luis pedro maria; do
    echo "Creando usuario: $usuario"
    # useradd "$usuario"
done
```

```
for archivo in /var/log/*.log; do
    echo "Procesando: $archivo"
    # gzip "$archivo"
done
```

6.2 Bucle While

Ejecuta mientras la condición sea verdadera:

```
#!/bin/bash
CONTADOR=1
while [ $CONTADOR -le 5 ]; do
    echo "Iteración $CONTADOR"
    CONTADOR=$((CONTADOR + 1))
done
```

6.3 Control de bucles

- **break**: sale del bucle inmediatamente.
- **continue**: salta a la siguiente iteración.

```
for num in {1..10}; do
    if [ $num -eq 5 ]; then
        continue  # Salta el 5
    fi
    if [ $num -eq 8 ]; then
        break     # Se detiene al llegar a 8
    fi
    echo $num
done
```

7. FUNCIONES

Las funciones permiten organizar y reutilizar código:

7.1 Definición y llamada

```bash
# Definición
saludar() {
    echo "Hola, $1"
}
```

```bash
# Llamada
saludar "Mundo"
```

7.2 Parámetros

Las funciones reciben argumentos igual que los scripts: $1, $2, etc. La variable $0 dentro de una función sigue siendo el nombre del script.

```bash
crear_usuario() {
    local nombre="$1"
    local grupo="$2"
    echo "Creando $nombre en grupo $grupo"
    # useradd -m -g "$grupo" "$nombre"
}
```

```bash
crear_usuario "ana" "profesores"
```

7.3 Retorno de valores

Las funciones pueden "devolver" valores imprimiéndolos y capturándolos:

```bash
contar_archivos() {
    local dir="$1"
    ls "$dir" | wc -l
}
```

```bash
TOTAL=$(contar_archivos /etc)
echo "Hay $TOTAL archivos en /etc"
```

8. PLANTILLA DE SCRIPT PROFESIONAL

Con el objetivo de mantener la misma filosofía de trabajo entre todos los alumnos del grupo, se va a utilizar la siguiente plantilla genérica para todos los script a implementar:

8.1 Descripción o cabecera del script

```
#!/bin/bash
#
# Nombre: gestion_usuarios.sh
# Descripción: Script para gestión de usuarios del sistema
# Uso: ./gestion_usuarios.sh {crear|eliminar} usuario
# Autor: Alumno ASIR
# Fecha: 2025-01-15
#
```

8.2 Funciones

Con la definición de las funciones que contiene todo el código repetitivo del script:

```
# --- Funciones ---
mostrar_uso() {
    echo "Uso: $0 {crear|eliminar} nombre_usuario"
    exit 1
}
```

8.3 Zona de Validación

Con las validaciones y comprobaciones previas a realizar antes de ejecutar el código principal asociado al script.

```
# --- Principal ---
[ $# -ne 2 ] && mostrar_uso

case "$1" in
    crear)
```

8.4 Principal

Implementación de la lógica de negocio del script.

```
# --- Principal ---
[ $# -ne 2 ] && mostrar_uso

case "$1" in
    crear)
        crear_usuario "$2"
        ;;
```